

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: AI-Powered Drone Delivery in a Dynamic Flood-Affected Environment

Scenario Analysis

In disaster-stricken regions, rapid delivery of emergency medicines is critical for saving lives. The government deploys autonomous AI-powered drones to transport medical supplies across flood-affected areas where conventional transportation infrastructure has been severely disrupted. Unlike a static navigation problem, the operational environment is highly dynamic because roads may become inaccessible without prior warning, weather conditions continuously influence flight safety and energy consumption, and new information regarding blocked routes is received during mission execution.

From an Artificial Intelligence perspective, this problem represents a **dynamic path-planning problem** in which an intelligent agent must continuously determine an optimal path while minimizing delivery time, operational risk, and resource consumption. Since the search space changes over time, the efficiency of the underlying search algorithm directly impacts mission success.

(i) Behaviour of Greedy Best-First Search (GBFS) in the Given Scenario

Greedy Best-First Search (GBFS) is an informed search algorithm that expands the node appearing closest to the goal according to a heuristic function. Unlike cost-based search algorithms, GBFS ignores the accumulated path cost and relies solely on the heuristic estimate.

The evaluation function of Greedy Best-First Search is

$$f(n) = h(n)$$

where

- $h(n)$ represents the heuristic estimate of the remaining distance from the current node to the destination.

In the given scenario, the heuristic may be the straight-line (Euclidean) distance between the drone's current position and the emergency delivery location. At every decision point, the drone evaluates neighbouring locations and selects the node having the minimum heuristic value.

For example, assume the drone has three possible routes:

Route	Actual Cost	Heuristic Distance
Route A	18	4 km
Route B	12	6 km
Route C	15	5 km

Since GBFS considers only the heuristic value, it immediately selects Route A because it appears closest to the destination, even though Route B requires significantly lower travel cost.

Strengths of GBFS

The primary advantage of GBFS is its computational efficiency. Since only heuristic information is evaluated, fewer nodes are expanded compared to uninformed search techniques. Consequently, the algorithm provides extremely fast decision-making, making it attractive for emergency response applications where immediate action is more important than exhaustive exploration.

Additional advantages include:

- Low computational complexity due to selective node expansion.

- Reduced memory consumption because fewer states are maintained.
- Fast response suitable for real-time navigation.
- Easy implementation using priority queues.

Weaknesses under Uncertainty

Despite its speed, GBFS performs poorly in highly dynamic environments because it completely ignores the accumulated travel cost.

In flood conditions, the route with the shortest apparent distance may actually contain:

- flooded roads,
- severe weather disturbances,
- temporary no-fly zones,
- high wind resistance,
- damaged infrastructure.

Since GBFS lacks awareness of these operational costs, it may repeatedly choose unsafe or inefficient routes.

Furthermore, because the algorithm is neither complete nor optimal in general search spaces, it may become trapped in local minima or repeatedly explore misleading paths generated by inaccurate heuristic estimates.

Therefore, although GBFS minimizes computational latency, its path quality deteriorates significantly when environmental uncertainty increases.

(ii) Behaviour of A* Search in the Same Scenario

A* Search extends informed search by simultaneously considering the actual traversal cost and the heuristic estimate of the remaining distance. Instead of making purely greedy decisions, A* evaluates the total estimated cost required to reach the destination.

The evaluation function is

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ is the accumulated cost from the start node to the current node.
- $h(n)$ is the heuristic estimate from the current node to the goal.

The value $f(n)$ represents the estimated total cost of reaching the destination.

During operation, the drone continuously performs the following steps:

1. Compute the accumulated travel cost.
2. Estimate the remaining distance.
3. Calculate the total evaluation function.
4. Expand the node having the minimum $f(n)$.
5. Update costs whenever new environmental information becomes available.

Unlike GBFS, A* considers weather severity, terrain complexity, battery consumption, and blocked routes as components of the actual travel cost. Consequently, the selected route may not be geographically shortest but is generally safer and operationally more efficient.

For instance:

Route $g(n)$ $h(n)$ $f(n)$

Route A 18 4 22

Route B 12 6 18

Route C 15 5 20

Although Route A has the smallest heuristic value, A* correctly selects Route B because it minimizes the total estimated delivery cost.

This balanced evaluation significantly improves navigation quality in disaster management systems.

(iii) Impact of Heuristic Accuracy on GBFS and A*

The effectiveness of both algorithms depends on the quality of the heuristic function.

An accurate heuristic closely approximates the true remaining cost. Under such conditions:

- fewer nodes are expanded,
- computational overhead decreases,
- search convergence becomes faster,
- solution quality improves.

However, the impact differs considerably between GBFS and A*.

Effect on Greedy Best-First Search

Since GBFS depends exclusively on $h(n)$, any error in heuristic estimation directly affects decision quality.

An overestimated or misleading heuristic may cause the drone to:

- select dangerous routes,
- ignore safer alternatives,
- revisit unnecessary states,
- produce highly suboptimal solutions.

Therefore, GBFS exhibits high sensitivity to heuristic inaccuracies.

Effect on A*

A* is comparatively robust because the accumulated path cost compensates for moderate heuristic errors.

Even if the heuristic is imperfect, the algorithm still evaluates the actual traversal cost before expanding nodes.

When an admissible heuristic (one that never overestimates the true remaining cost) is used, A* guarantees an optimal solution.

Thus, heuristic quality influences computational efficiency, whereas optimality remains preserved under admissible conditions.

(iv) Impact of Dynamic Changes such as Blocked Paths

Flood environments are characterized by continuous environmental changes.

Examples include:

- collapsed bridges,
- newly flooded roads,
- severe weather alerts,
- changing wind directions,
- restricted airspace.

These dynamic events invalidate previously computed paths.

Effect on Greedy Best-First Search

GBFS lacks a mechanism for effectively incorporating changing traversal costs. When a selected route becomes blocked, the algorithm often restarts the search from the current position, resulting in redundant computations and increased delivery delays.

Repeated replanning also increases the likelihood of oscillating between neighbouring nodes when heuristic estimates remain similar.

Effect on A*

A* manages dynamic environments more effectively because path costs are explicitly maintained.

Whenever a blockage is detected:

- edge costs are updated,
- affected nodes are re-evaluated,
- alternative routes are explored,

- the lowest total cost path is reconstructed.

Although replanning introduces additional computation, the resulting path remains safer and more reliable.

Therefore, A* demonstrates superior adaptability in real-time disaster response operations.

(v) Recommendation and Justification

Considering the operational requirements of emergency medical delivery, *A Search is the most appropriate algorithm*.*.

The justification is based on the following factors:

Real-Time Decision Making: Although GBFS provides faster initial decisions, A* maintains acceptable response times while producing significantly better routes.

Optimality: A* guarantees the optimal path when an admissible heuristic is employed, whereas GBFS offers no such guarantee.

Safety: By incorporating actual traversal costs, A* naturally avoids hazardous routes affected by flooding, adverse weather, and infrastructure damage.

Adaptability: The algorithm effectively integrates updated environmental information, enabling continuous replanning without sacrificing solution quality.

Mission Reliability: In humanitarian operations, delivery accuracy and safety are considerably more important than minimal computation time. A* achieves a practical balance between computational efficiency and operational reliability.

Hence, despite requiring greater memory and processing resources than Greedy Best-First Search, A* provides the highest probability of successful medicine delivery in dynamic disaster environments.

Conclusion

The drone navigation problem represents a dynamic informed search problem requiring rapid yet reliable decision-making under uncertainty. Greedy Best-First Search offers low computational overhead and fast route selection but performs poorly when environmental conditions change because it ignores accumulated path costs. In contrast, A* integrates both actual traversal cost and heuristic guidance, enabling the drone to identify routes that minimize overall delivery cost while maintaining safety and adaptability. Consequently, A* is the preferred search strategy for autonomous emergency drone systems operating in dynamic and safety-critical environments.¹

Question 2: Smart City Traffic Signal Optimization using Hill Climbing, Simulated Annealing, and Genetic Algorithms

Scenario Analysis

Modern smart cities employ Artificial Intelligence (AI) to optimize traffic signal timings across hundreds of intersections with the objective of minimizing traffic congestion, reducing vehicle waiting time, and lowering fuel consumption. Unlike conventional traffic control systems that operate on predefined schedules, AI-based traffic management systems continuously adapt signal timings using real-time traffic data obtained from cameras, IoT sensors, GPS-enabled vehicles, and roadside units.

This problem represents a **combinatorial optimization problem**, where the AI system must determine the optimal signal timing configuration from an enormous number of possible combinations. Since traffic conditions fluctuate continuously due to accidents, weather conditions, road closures, and peak-hour demand, the optimization algorithm must efficiently

search a large solution space while adapting to changing environments. Traditional search algorithms become computationally infeasible because the number of possible signal configurations increases exponentially with the number of intersections. Therefore, optimization algorithms such as **Hill Climbing**, **Simulated Annealing**, and **Genetic Algorithms** are preferred due to their ability to efficiently explore large search spaces and produce high-quality solutions within acceptable computational time.

Problem Formulation as an Optimization Problem

The traffic signal optimization problem can be mathematically formulated as follows.

Objective Function

The AI system aims to minimize the following cost function:

$$F = w_1(T_w) + w_2(F_c) + w_3(C)$$

Where:

- T_w = Total vehicle waiting time
- F_c = Fuel consumption
- C = Traffic congestion level
- w_1, w_2, w_3 = Weight factors assigned according to system priorities

The optimization objective is

$$\min F$$

subject to operational constraints such as:

- Minimum and maximum green signal duration.
- Pedestrian crossing intervals.
- Emergency vehicle priority.
- Traffic regulations.
- Intersection synchronization.

Each possible traffic signal configuration represents a **state** in the search space, while changing the duration of one or more traffic signals represents an **operator** that generates neighbouring states. Since the number of possible timing combinations grows exponentially with the number of intersections, exhaustive search is computationally impractical for real-time applications.

Why Traditional Search is Inefficient

Traditional search algorithms, such as Breadth-First Search (BFS) and Depth-First Search (DFS), systematically explore every possible state until an acceptable solution is found. Although these algorithms guarantee completeness under certain conditions, they are unsuitable for large-scale optimization problems.

In a metropolitan traffic network containing hundreds of intersections, each signal may have multiple timing options. Consequently, the total number of possible configurations becomes extremely large, resulting in a combinatorial explosion. Exploring every configuration would require excessive computational time and memory, making real-time optimization impossible. Furthermore, traffic patterns continuously evolve due to external factors such as vehicle density, accidents, weather changes, and special events. Traditional search assumes a relatively static environment and therefore cannot efficiently respond to continuously changing traffic conditions. Even if an optimal solution is eventually discovered, it may become obsolete before implementation.

Hence, intelligent optimization techniques capable of producing near-optimal solutions within limited computational time are required for practical smart city traffic management.

(i) Hill Climbing Algorithm in Traffic Signal Optimization

Hill Climbing is a heuristic optimization algorithm that iteratively improves the current solution by moving toward neighbouring solutions with better objective values. It follows a greedy strategy by selecting the immediate improvement without considering future possibilities.

In the context of traffic signal optimization, the algorithm begins with an initial signal timing schedule and evaluates its performance based on the objective function. Small modifications, such as increasing or decreasing the green signal duration at a particular intersection, generate neighbouring solutions. If a neighbouring solution reduces congestion, waiting time, or fuel consumption, it replaces the current solution. This iterative improvement continues until no neighbouring solution provides further improvement.

Working Procedure

1. Initialize traffic signal timings.
2. Evaluate the objective function.
3. Generate neighbouring signal configurations.
4. Select the neighbour with the greatest improvement.
5. Replace the current solution if improvement exists.
6. Terminate when no better neighbour is available.

Since Hill Climbing always moves toward an improving solution, it converges rapidly and requires relatively little computational memory.

Advantages

- Simple algorithm with straightforward implementation.
- Low computational and memory requirements.
- Rapid convergence for small optimization problems.
- Suitable for real-time applications where quick decisions are required.

Limitations

Despite its efficiency, Hill Climbing has several significant drawbacks.

- It may converge to a local optimum instead of the global optimum.
- It cannot escape plateaus where neighbouring solutions have identical objective values.
- It performs poorly in complex search landscapes containing ridges.
- The final solution depends heavily on the initial starting point.

Therefore, Hill Climbing alone cannot guarantee globally optimal traffic signal timings in large metropolitan road networks.

(ii) Local Maxima, Plateaus, and Ridges: Real-World Interpretation

The search landscape associated with traffic optimization is highly irregular and contains numerous local optima. Three common challenges encountered by Hill Climbing are local maxima, plateaus, and ridges.

Local Maxima

A local maximum represents a solution that is superior to all neighbouring solutions but is not the best solution within the entire search space.

In traffic optimization, the AI system may identify a signal configuration that significantly reduces congestion in one district while inadvertently increasing congestion elsewhere. Since no neighbouring configuration appears superior, the algorithm prematurely terminates despite the existence of a much better global solution.

Plateaus

A plateau is a region where multiple neighbouring solutions possess nearly identical objective values. Consequently, the algorithm receives no information indicating which direction should be explored.

For example, adjusting signal timings by a few seconds during periods of light traffic may produce almost identical waiting times across multiple intersections. Since all neighbouring configurations appear equally effective, the search process stagnates.

Ridges

A ridge represents a narrow path toward the global optimum that requires multiple coordinated modifications rather than a single improvement.

In an urban traffic network, reducing congestion along a major corridor often requires synchronized adjustments across several adjacent intersections. Individual modifications may appear ineffective, preventing Hill Climbing from progressing toward the optimal configuration.

These challenges significantly reduce the effectiveness of simple local search techniques in large-scale optimization problems.

(iii) Overcoming Limitations using Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing is an advanced local search algorithm inspired by the metallurgical annealing process. Unlike Hill Climbing, it occasionally accepts inferior solutions with a controlled probability, enabling exploration beyond local optima.

Initially, the algorithm operates at a high "temperature," allowing large exploratory movements throughout the search space. As the temperature gradually decreases, the probability of accepting inferior solutions diminishes, causing the algorithm to converge toward high-quality solutions.

For traffic optimization, this behaviour enables the AI system to temporarily accept slightly worse traffic signal configurations, thereby escaping local maxima and discovering superior global solutions.

Advantages

- Escapes local optima.
- Handles complex search landscapes effectively.
- Produces better global solutions than Hill Climbing.
- Suitable for dynamic optimization problems.

However, solution quality depends significantly on the cooling schedule, and convergence may require considerable computational time.

Genetic Algorithm

Genetic Algorithms (GA) are population-based optimization techniques inspired by biological evolution. Rather than improving a single solution, GA simultaneously evolves multiple candidate solutions across successive generations.

Each chromosome represents a complete traffic signal schedule, while the fitness function evaluates its effectiveness based on congestion, waiting time, and fuel consumption.

The optimization process involves four primary stages:

- **Selection:** High-performing schedules are selected for reproduction.
- **Crossover:** Parent solutions exchange timing information to generate offspring.
- **Mutation:** Random modifications introduce diversity and prevent premature convergence.
- **Replacement:** Inferior solutions are replaced by higher-quality offspring.

Through repeated evolutionary cycles, the population gradually converges toward globally optimal or near-optimal traffic signal configurations.

Advantages

- Efficient exploration of enormous search spaces.
- Reduced likelihood of becoming trapped in local optima.

- Naturally supports multi-objective optimization.
- Highly scalable for metropolitan traffic networks.
- Easily parallelized for high-performance computing environments.

Although Genetic Algorithms require greater computational resources than Hill Climbing, their superior optimization capability makes them highly effective for large-scale smart city applications.

(iv) Recommended Approach and Justification

Among the three optimization techniques, the **Genetic Algorithm** is the most appropriate choice for large-scale smart city traffic signal optimization.

The recommendation is justified by the following considerations:

Scalability

Genetic Algorithms efficiently manage hundreds of intersections simultaneously, whereas Hill Climbing becomes increasingly ineffective as the search space expands.

Adaptability

By continuously evolving candidate solutions using updated traffic information, GA effectively adapts to changing traffic patterns, accidents, and unexpected congestion.

Global Optimization Capability

Unlike Hill Climbing, which frequently converges to local optima, Genetic Algorithms perform global exploration through crossover and mutation, significantly increasing the probability of identifying superior traffic signal configurations.

Multi-Objective Optimization

Traffic management simultaneously seeks to minimize waiting time, fuel consumption, and congestion. Genetic Algorithms naturally optimize multiple objectives by incorporating them into a comprehensive fitness function.

Practical Applicability

Several intelligent transportation systems employ evolutionary optimization techniques because they produce high-quality solutions within acceptable computational time while maintaining flexibility for real-time adaptation.

Conclusion

The smart city traffic signal optimization problem represents a complex, dynamic, and large-scale optimization challenge that cannot be efficiently solved using traditional search algorithms due to the exponential growth of the search space. Although Hill Climbing provides rapid convergence through local improvements, it suffers from limitations such as local maxima, plateaus, and ridges, preventing it from consistently identifying globally optimal solutions. Simulated Annealing addresses these limitations by probabilistically accepting inferior solutions, thereby improving exploration of the search space. However, the **Genetic Algorithm** offers the most robust and scalable solution by simultaneously evolving multiple candidate signal configurations through evolutionary operators such as selection, crossover, and mutation. Its superior adaptability, scalability, and global optimization capability make it the most suitable algorithm for real-time traffic signal optimization in modern smart city environments.

Question 3: Autonomous Mars Rover Navigation Using an Online Search Agent

Scenario Analysis

Planetary exploration presents one of the most challenging applications of Artificial Intelligence due to the absence of complete environmental knowledge and the inability to rely on continuous human intervention. In this scenario, an autonomous Mars rover is deployed to explore an unknown planetary surface with the objectives of collecting geological samples, avoiding hazardous terrain such as craters and rocks, and navigating safely despite possessing only limited sensing capabilities. Unlike terrestrial navigation systems that operate using detailed maps and real-time human guidance, the rover must independently perceive its surroundings, update its knowledge of the environment, and make navigation decisions in real time.

From an Artificial Intelligence perspective, this problem represents an **online search problem** operating within a **partially observable and dynamic environment**. The rover cannot compute an optimal path before execution because the terrain is initially unknown. Instead, it must simultaneously explore the environment, gather information through onboard sensors, construct an internal representation of the terrain, and continuously revise its navigation strategy. Consequently, intelligent decision-making under uncertainty becomes essential for ensuring mission success while minimizing operational risks.

(i) Why the Problem Requires an Online Search Agent Instead of Offline Search

Artificial Intelligence search strategies are broadly classified into **offline search** and **online search** based on the availability of environmental information before problem solving. Offline search assumes that the intelligent agent possesses complete knowledge of the environment prior to initiating the search process. The complete state space, possible actions, transition model, and goal location are known in advance, enabling the algorithm to compute an optimal path before execution.

However, the Mars rover scenario violates these assumptions. At the beginning of the mission, the rover has no complete map of the terrain and possesses only local sensory information obtained through cameras, LiDAR, radar, and proximity sensors. Environmental conditions such as loose rocks, sand dunes, steep slopes, and craters become observable only when the rover physically approaches them. Consequently, the rover cannot precompute a complete navigation path.

An **Online Search Agent** addresses this limitation by integrating exploration and decision-making. Instead of planning an entire route beforehand, the rover incrementally explores unknown regions, updates its environmental knowledge, and determines the next action based on its current observations.

The operational cycle of the online search agent consists of the following stages:

1. Sense the surrounding environment using onboard sensors.
2. Update the internal environmental model.
3. Evaluate possible actions.
4. Select the safest and most efficient movement.
5. Execute the selected action.
6. Repeat the process until the exploration objective is achieved.

Unlike offline search, online search continuously adapts to newly acquired information, making it the most suitable approach for autonomous planetary exploration.

(ii) Challenges of Partial Observability and Dynamic Environments

The Mars exploration environment presents several uncertainties that significantly influence the rover's navigation strategy.

Partial Observability

Partial observability refers to situations in which the intelligent agent has access to only a limited portion of the environment at any given time. The rover cannot observe distant terrain beyond the range of its sensors, and therefore must make decisions using incomplete information.

Examples include:

- Hidden craters located beyond hills.
- Rocks concealed by dust accumulation.
- Slopes whose inclination cannot be accurately estimated from a distance.
- Unknown terrain composition affecting wheel traction.

Since future states remain unknown, the rover must continuously revise its navigation strategy as additional information becomes available.

This uncertainty introduces several computational challenges:

- Increased decision complexity.
- Higher probability of selecting unsafe paths.
- Frequent replanning.
- Greater computational requirements for environmental modelling.

Dynamic Environment

Although planetary terrain changes relatively slowly compared to urban environments, Mars still exhibits several dynamic characteristics that influence navigation.

Examples include:

- Dust storms reducing sensor visibility.
- Sand displacement caused by wind.
- Temperature variations affecting sensor accuracy.
- Wheel slippage due to unstable soil.
- Unexpected obstacles discovered during movement.

These continuously changing conditions invalidate previously planned routes and require the rover to adapt its decisions in real time.

Consequently, static search strategies become ineffective because previously optimal paths may become hazardous after environmental changes.

(iii) Internal Model Construction and Continuous Knowledge Updating

An intelligent autonomous rover relies on an **internal world model** to represent its current understanding of the surrounding environment. Since complete environmental knowledge is unavailable at mission initiation, this model evolves continuously throughout exploration. Initially, the internal model contains only the rover's starting position and mission objectives. As exploration progresses, information obtained through cameras, LiDAR sensors, inertial measurement units (IMU), GPS alternatives, and obstacle detection systems is incorporated into the model.

The knowledge updating process consists of the following stages:

1. Environment Perception

The rover continuously scans its surroundings using multiple sensors to detect terrain features, obstacles, and navigable paths.

2. Feature Extraction

Raw sensor data is processed to identify significant environmental characteristics such as rocks, craters, slopes, and scientifically valuable geological formations.

3. Map Construction

The extracted information is integrated into an internal occupancy grid or topological map representing explored and unexplored regions.

4. State Updating

Previously unknown areas become classified according to newly acquired information, enabling the rover to distinguish between safe, hazardous, and unexplored regions.

5. Path Replanning

Whenever environmental changes are detected, the rover recomputes its navigation strategy using the updated world model.

This continuous learning mechanism enables the rover to improve navigation accuracy throughout the mission while minimizing collision risks.

(iv) Comparison of Online Search with Uninformed and Informed Search Strategies

Different search strategies exhibit distinct characteristics when applied to planetary exploration.

Criteria	Uninformed Search	Informed Search	Online Search
Environmental Knowledge	No prior knowledge	Requires heuristic information	Learns during exploration
Complete Map Required	Yes	Yes	No
Uses Heuristic Function	No	Yes	Optional
Adaptation to New Information	Very Limited	Limited	Excellent
Suitable for Unknown Terrain	No	Partially	Yes
Decision Making	Fixed before execution	Mostly pre-planned	Continuous and adaptive
Computational Efficiency	Low	Moderate	High in uncertain environments
Real-Time Navigation	Poor	Moderate	Excellent

Uninformed search algorithms such as Breadth-First Search and Depth-First Search assume that the environment is fully known before search execution, making them unsuitable for unexplored planetary surfaces.

Informed search algorithms such as A* improve efficiency by utilizing heuristic information; however, they still require an initially available map for effective path planning. When the environment continuously changes, repeated replanning becomes computationally expensive. Online Search overcomes these limitations by integrating perception, learning, and decision-making into a unified framework. The rover incrementally builds environmental knowledge while simultaneously navigating toward mission objectives.

(v) Recommendation and Justification

Considering the uncertainty associated with planetary exploration, the **Online Search Agent** is the most appropriate Artificial Intelligence approach for autonomous Mars rover navigation.

The recommendation is justified based on the following technical considerations.

Adaptability

The rover continuously incorporates newly observed information into its navigation strategy, enabling effective operation within unknown environments.

Handling Uncertainty

Unlike offline search algorithms, online search naturally manages incomplete environmental information and does not require a fully known state space before execution.

Real-Time Decision Making

Navigation decisions are computed dynamically according to current environmental conditions, allowing immediate responses to newly detected obstacles and hazards.

Safety

Continuous obstacle detection and route replanning significantly reduce collision probability and improve mission reliability.

Scalability

The online search framework remains effective regardless of exploration area size because only locally available information is processed at each decision step.

Mission Reliability

By continuously updating its internal world model and adapting navigation decisions accordingly, the rover maximizes exploration efficiency while minimizing operational risks and energy consumption.

Therefore, Online Search provides the highest level of autonomy, flexibility, and robustness for intelligent planetary exploration systems.

Conclusion

The autonomous Mars rover navigation problem represents a complex real-time decision-making task characterized by partial observability, environmental uncertainty, and the absence of prior terrain knowledge. Traditional offline search algorithms are unsuitable because they require complete environmental information before planning begins, while informed search techniques depend on reliable heuristic guidance derived from known maps. In contrast, an **Online Search Agent** continuously integrates perception, learning, and navigation by constructing and updating an internal environmental model throughout exploration. Its ability to adapt to newly discovered obstacles, revise navigation strategies in real time, and safely operate under uncertain conditions makes it the most appropriate Artificial Intelligence approach for autonomous planetary exploration missions.

Question 4: University Examination Timetable Generation Using Constraint Satisfaction Problem (CSP)

Scenario Analysis

Universities conduct examinations involving thousands of students, hundreds of courses, multiple examination halls, and numerous faculty members. Preparing an examination timetable manually is a complex and time-consuming process because several academic and administrative constraints must be satisfied simultaneously. In the given scenario, the university must ensure that no student has overlapping examinations, examination halls are allocated efficiently, faculty members are available for invigilation, prerequisite subjects are scheduled appropriately, and special accommodations are provided for students with disabilities. As the number of courses and enrolled students increases, the scheduling problem becomes computationally more challenging.

From an Artificial Intelligence perspective, examination scheduling is a **Constraint Satisfaction Problem (CSP)** because the objective is not to optimize a numerical value but to determine a feasible assignment of variables that satisfies a predefined set of constraints.

Unlike conventional search problems that seek the shortest path or minimum cost, CSP focuses on assigning values to variables while ensuring that all constraints are simultaneously satisfied. Therefore, AI techniques such as **Backtracking Search** and **Constraint Propagation** are widely employed to solve large-scale scheduling problems efficiently.

Problem Formulation as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem is formally represented by three fundamental components:

- **Variables (X)**
- **Domains (D)**
- **Constraints (C)**

The objective is to assign a value from the domain to every variable while satisfying all constraints.

Variables

Variables represent the entities that require assignment.

In the examination scheduling problem, each course is considered a variable.

For example,

$$X = \{AI, DBMS, OS, CN, ML, CC\}$$

where

- AI = Artificial Intelligence
- DBMS = Database Management Systems
- OS = Operating Systems
- CN = Computer Networks
- ML = Machine Learning
- CC = Cloud Computing

Each subject must be assigned an appropriate examination slot.

Domains

The domain represents all possible values that each variable can assume.

For every examination subject, the domain consists of:

- Available examination dates
- Available time slots
- Available examination halls

Example:

Subject Possible Domain

AI Monday FN, Monday AN, Tuesday FN

DBMS Monday FN, Tuesday AN, Wednesday FN

OS Tuesday FN, Wednesday AN

ML Thursday FN, Friday AN

Each subject can only be assigned one valid combination of date, time, and hall.

Constraints

Constraints specify the conditions that every assignment must satisfy.

They are generally classified into **Hard Constraints** and **Soft Constraints**.

Hard Constraints

These constraints must always be satisfied.

Examples include:

- A student cannot have two examinations at the same time.
- One examination hall cannot accommodate more students than its capacity.
- A faculty member cannot invigilate two examinations simultaneously.

- Certain subjects must be conducted before advanced subjects.
- Students requiring special accommodations must be allocated appropriate examination halls.
- Every examination hall can host only one examination during a particular time slot.

Violation of any hard constraint produces an invalid timetable.

Soft Constraints

Soft constraints improve timetable quality but are not mandatory.

Examples include:

- Avoid scheduling difficult subjects consecutively.
- Provide adequate preparation gaps between examinations.
- Balance faculty workload.
- Minimize unused examination hall capacity.
- Schedule examinations according to departmental preferences.

Although violating soft constraints does not invalidate the timetable, satisfying them significantly improves scheduling quality.

(ii) Backtracking Search in Examination Scheduling

Backtracking Search is the most widely used complete search algorithm for solving Constraint Satisfaction Problems. It incrementally constructs a solution by assigning values to variables while continuously checking constraint satisfaction.

The algorithm begins by selecting an unassigned examination subject and assigning one of the available time slots from its domain. After every assignment, all constraints are verified. If no constraint is violated, the algorithm proceeds to schedule the next subject. However, if a conflict occurs, the algorithm backtracks by removing the conflicting assignment and attempting another available value.

The process continues recursively until either a complete valid timetable is generated or all possible assignments have been exhausted.

Working Procedure

1. Select an unassigned examination subject.
2. Choose an available date, time slot, and examination hall.
3. Verify all scheduling constraints.
4. If constraints are satisfied, assign the value permanently.
5. Continue scheduling the remaining subjects.
6. If a conflict occurs, undo the previous assignment.
7. Try an alternative assignment.
8. Repeat until every subject has been successfully scheduled.

Illustrative Example

Suppose the university has four examinations:

- AI
- DBMS
- Operating Systems
- Machine Learning

Initially,

AI → Monday Morning ✓

Next,

DBMS → Monday Morning ✗

Conflict:

Several students are enrolled in both AI and DBMS.

The algorithm immediately detects the violation.

Backtracking occurs.

DBMS → Tuesday Morning ✓

Scheduling continues until all examinations satisfy every constraint.

Advantages of Backtracking

- Guarantees a complete solution if one exists.
- Ensures all hard constraints are satisfied.
- Simple recursive implementation.
- Applicable to various CSP problems.

Limitations

Despite its correctness, Backtracking suffers from several computational limitations.

- Large search space.
- High computational complexity.
- Frequent backtracking in complex timetables.
- Increased execution time as the number of courses grows.

Therefore, additional optimization techniques are required to improve efficiency.

(iii) Constraint Propagation (Forward Checking) and Its Role in Improving Efficiency

Constraint Propagation is an inference technique that reduces the search space before conflicts occur. Instead of waiting until an invalid assignment is discovered, the algorithm proactively eliminates inconsistent values from the domains of unassigned variables.

One of the most effective constraint propagation techniques is **Forward Checking**.

Whenever a variable receives an assignment, Forward Checking immediately removes conflicting values from the domains of neighbouring variables.

Working of Forward Checking

Assume:

AI Examination

→ Monday Morning

Immediately after this assignment,

Forward Checking performs the following operations:

- Removes Monday Morning from every subject having common students with AI.
- Removes examination halls already allocated.
- Updates faculty availability.
- Removes conflicting invigilation assignments.

Consequently, subsequent variables possess smaller domains, reducing the probability of future conflicts.

Example

Initially,

DBMS Domain

{Monday FN, Tuesday FN, Wednesday FN}

After assigning

AI → Monday FN

Updated Domain

{Tuesday FN, Wednesday FN}

The conflicting value is removed before scheduling begins.

This significantly reduces unnecessary backtracking.

Benefits of Constraint Propagation

- Reduces search space.
- Detects inconsistencies early.
- Minimizes recursive backtracking.
- Improves computational efficiency.
- Accelerates timetable generation.
- Produces valid schedules with fewer iterations.

Constraint Propagation becomes increasingly valuable as the number of students and courses increases because it prevents the exploration of impossible scheduling combinations.

(iv) Real-World Challenges and Best Strategy for Scalability

Modern universities face numerous practical challenges during examination scheduling.

Large Student Population

Thousands of students enroll in multiple interdisciplinary courses, creating highly interconnected scheduling dependencies.

Limited Infrastructure

The number of available examination halls is limited, requiring efficient resource allocation while maintaining seating capacity constraints.

Faculty Availability

Faculty members have teaching schedules, research commitments, and administrative responsibilities that restrict invigilation assignments.

Special Accommodation Requirements

Students with disabilities require additional facilities such as accessible examination halls, scribes, or extended examination durations.

Dynamic Schedule Modifications

Unexpected events, including faculty absence, examination postponements, or emergency situations, may require timetable revisions after initial generation.

Recommended Strategy

The most effective approach is the combination of **Backtracking Search** with **Constraint Propagation (Forward Checking)**.

This integrated strategy offers several advantages:

Efficient Search

Backtracking systematically explores valid assignments while guaranteeing completeness.

Reduced Computational Cost

Forward Checking eliminates inconsistent assignments before recursive exploration begins, significantly reducing unnecessary computations.

Scalability

The combined approach remains effective even when scheduling hundreds of courses and thousands of students.

Constraint Satisfaction

Both hard and soft constraints can be incorporated without compromising timetable validity.

Practical Implementation

Most modern university timetable generation systems employ CSP solvers enhanced with constraint propagation because they provide reliable, scalable, and computationally efficient scheduling solutions.

Therefore, the integration of **Backtracking Search** and **Constraint Propagation** represents the most suitable AI strategy for automated examination timetable generation.

Conclusion

The university examination scheduling problem is a classic **Constraint Satisfaction Problem (CSP)** in which examination subjects are modeled as variables, available dates, time slots, and examination halls constitute the domains, and academic regulations are represented as constraints. Although **Backtracking Search** guarantees a valid timetable by systematically exploring feasible assignments, its computational performance deteriorates as the search space expands. **Constraint Propagation**, particularly **Forward Checking**, significantly enhances efficiency by eliminating inconsistent domain values before conflicts arise, thereby reducing unnecessary backtracking. The combined application of Backtracking Search and Constraint Propagation provides a scalable, reliable, and computationally efficient solution capable of generating conflict-free examination timetables for large universities while satisfying complex academic and administrative constraints.

Question 5: Strategic Game AI for Real-Time Decision Making Using Minimax and Alpha-Beta Pruning

Scenario Analysis

Modern Real-Time Strategy (RTS) games require Artificial Intelligence (AI) agents capable of making intelligent decisions under strict time constraints while competing against skilled human players. Unlike turn-based games with relatively small search spaces, RTS games involve thousands of possible actions at every decision point, including resource management, unit movement, combat strategies, defensive planning, and territory expansion. Consequently, the decision tree grows exponentially, making exhaustive evaluation computationally infeasible.

The AI opponent must continuously analyze the current game state, predict the possible responses of the human player, and select actions that maximize its probability of winning. Since every decision influences future game states, the problem is naturally modeled as an **adversarial search problem**, where one player's gain corresponds to the opponent's loss. Artificial Intelligence addresses this challenge through the **Minimax algorithm**, which models rational decision-making between two competing agents. To overcome the computational complexity associated with large game trees, **Alpha-Beta Pruning** is integrated with Minimax to eliminate branches that cannot influence the final decision, thereby significantly improving search efficiency.

(i) How the Minimax Algorithm Models the Problem

Minimax is an adversarial search algorithm designed for two-player, zero-sum games in which one player's objective is to maximize the utility function while the opponent attempts to minimize it.

In the given scenario,

- The **AI opponent** acts as the **MAX player**, attempting to maximize its advantage.
- The **Human player** acts as the **MIN player**, attempting to minimize the AI's advantage.

The game is represented as a **search tree**, where:

- Each node represents a possible game state.
- Each edge represents a legal move.
- Leaf nodes contain utility values representing the desirability of terminal states.

The Minimax algorithm recursively explores the game tree by alternating between maximizing and minimizing levels until terminal states or a predefined search depth are reached.

The decision process follows these principles:

- **MAX nodes:** Select the move with the highest utility value.
- **MIN nodes:** Select the move with the lowest utility value.

The utility value is then propagated upward until the root node, allowing the AI to identify the optimal move assuming that the opponent always plays rationally.

Working Procedure

1. Generate all possible legal moves from the current game state.
2. Simulate each move to create successor states.
3. Alternate between MAX and MIN levels.
4. Continue until terminal states or depth limits are reached.
5. Evaluate terminal states using an evaluation function.
6. Propagate utility values upward through the tree.
7. Select the move with the highest utility value at the root.

By anticipating the opponent's responses, Minimax enables the AI to make strategically optimal decisions rather than merely reacting to the current game state.

(ii) Computational Challenges of Large Game Trees

Although Minimax guarantees optimal decision-making under perfect information, its practical implementation is limited by the exponential growth of the search tree.

In real-time strategy games, each player may have dozens or hundreds of possible actions at every decision point. Consequently, the branching factor becomes extremely large.

If the average branching factor is represented by **b** and the search depth by **d**, the time complexity of Minimax is

$$O(b^d)$$

This exponential complexity results in several computational challenges.

State Space Explosion

The number of possible game states increases exponentially as the game progresses.

For example,

- Unit movement
- Resource allocation
- Building construction
- Combat strategies
- Technology upgrades

all generate additional successor states.

High Computational Cost

Evaluating millions of game states within a few milliseconds exceeds the computational capabilities of most gaming systems.

Memory Consumption

The search tree requires substantial memory for storing intermediate game states, making deep searches impractical.

Strict Real-Time Constraints

Unlike board games where players may take several minutes to decide, RTS games require AI decisions within milliseconds.

Failure to produce timely decisions results in poor gameplay performance regardless of search quality.

Depth Limitation

Because complete tree exploration is impossible, practical implementations restrict search depth, preventing Minimax from evaluating long-term strategic consequences.

Therefore, optimization techniques become essential for maintaining real-time performance.

(iii) Alpha-Beta Pruning and Its Role in Improving Efficiency

Alpha-Beta Pruning is an optimization technique that significantly reduces the number of nodes explored by the Minimax algorithm without affecting the final decision.

Instead of evaluating every possible branch, Alpha-Beta identifies branches that cannot influence the optimal decision and eliminates them from further exploration.

The algorithm maintains two values throughout the search process.

Alpha (α)

Alpha represents the highest utility value that the MAX player can guarantee.

Initially,

$$\alpha = -\infty$$

As better moves are discovered, Alpha is updated accordingly.

Beta (β)

Beta represents the lowest utility value that the MIN player can guarantee.

Initially,

$$\beta = +\infty$$

Whenever a lower utility value is identified, Beta is updated.

Pruning Condition

Pruning occurs whenever

$$\alpha \geq \beta$$

Under this condition, further exploration of the remaining branches cannot improve the final decision because the opponent would never allow those states to occur.

Consequently, the algorithm immediately terminates exploration of that branch.

Illustrative Example

Consider a MAX node evaluating three possible moves.

Suppose the first move produces a utility value of **18**.

Therefore,

$$\alpha = 18$$

During evaluation of another branch, the MIN player identifies a node whose utility value becomes **12**.

Hence,

$$\beta = 12$$

Since

$$18 \geq 12$$

the remaining unexplored branches under that MIN node are pruned because they cannot influence the final decision.

This eliminates unnecessary computations while producing exactly the same result as the standard Minimax algorithm.

Advantages of Alpha-Beta Pruning

- Reduces the number of explored nodes.
- Produces identical decisions to Minimax.
- Enables deeper search within the same computational budget.
- Improves response time.
- Reduces memory requirements.
- Enhances real-time AI performance.

When optimal move ordering is employed, Alpha-Beta reduces the effective time complexity from

$$O(b^d)$$

to approximately

$$O(b^{d/2})$$

allowing significantly deeper game tree exploration.

(iv) Design of an Evaluation Function

Since complete exploration of the game tree is computationally infeasible, non-terminal game states must be evaluated using a heuristic evaluation function.

The evaluation function estimates the strategic quality of the current game state by combining multiple gameplay parameters.

An example evaluation function is

$$Score = 25(R) + 30(A) + 20(T) + 15(E) - 20(D) - 10(L)$$

where

- **R** = Available resources
- **A** = Army strength
- **T** = Territory controlled
- **E** = Economic development
- **D** = Damage sustained
- **L** = Strategic losses

Additional evaluation parameters may include:

- Unit health.
- Defensive fortifications.
- Resource production rate.
- Technological advancement.
- Number of surviving combat units.
- Enemy army strength.
- Map control.
- Distance to strategic objectives.

The weights assigned to each factor depend on the specific game mechanics and desired AI playing style.

A well-designed evaluation function enables the AI to approximate long-term strategic advantages without exhaustively exploring every possible future state.

(v) Recommendation for Balancing Optimality and Real-Time Performance

For real-time strategy games, the most effective AI architecture combines **Minimax**, **Alpha-Beta Pruning**, and **Heuristic Evaluation Functions**.

This recommendation is justified by the following considerations.

Optimal Decision Making

Minimax enables the AI to anticipate the opponent's future actions and select strategically advantageous moves.

Computational Efficiency

Alpha-Beta Pruning dramatically reduces unnecessary search, allowing the AI to evaluate deeper game trees within strict response time constraints.

Real-Time Adaptability

Heuristic evaluation functions estimate game state quality without requiring complete tree exploration, enabling timely decision-making even in highly complex game environments.

Scalability

The combined approach remains effective as the number of possible game states increases because pruning prevents exponential growth from becoming computationally prohibitive.

Industry Adoption

Most modern board-game and strategy-game engines employ Alpha-Beta Pruning together with heuristic evaluation functions because they provide an excellent balance between playing strength and computational efficiency.

Therefore, the integration of **Minimax**, **Alpha-Beta Pruning**, and **domain-specific heuristic evaluation** represents the most suitable AI strategy for developing intelligent real-time game opponents capable of making high-quality decisions under strict computational constraints.

Conclusion

The strategic game AI problem represents a classical adversarial search problem in which the AI must continuously maximize its probability of winning while assuming that the human opponent plays optimally. Although the **Minimax algorithm** provides an optimal decision-making framework, its exponential time complexity limits its practical application in large real-time strategy games. **Alpha-Beta Pruning** significantly improves computational efficiency by eliminating branches that cannot affect the final decision, thereby enabling deeper search within the same time constraints. When combined with an effective **heuristic evaluation function**, the AI can accurately assess non-terminal game states and make intelligent strategic decisions without exhaustively exploring the entire search tree.

Consequently, the integration of **Minimax**, **Alpha-Beta Pruning**, and **heuristic evaluation** provides the most efficient, scalable, and practically viable solution for real-time strategic game AI systems.